HW-SW Partitioning

James Kotary, Zakaria Mehrab, Rounak Meyur, Siddhartha Shankar Das,

May 2025

1 Experiments

1.1 Experimental Setup

The experiments are conducted on task graphs derived from real-world neural network models to ensure practical relevance. The task graphs are defined at tensor-level operations – check the last statement. Each task graph represents a directed acyclic graph (DAG) where nodes correspond to computational tasks and edges represent data dependencies with. Since the graphs are generated by ¡(ZM: Placeholder for task graph generation description)¿, we only obtain the topology. In order to test different partitioning methods on these task graphs, we have to assign costs associated with the nodes and edges. Extending Arato et al. [1], we employ the following mechanism for task graph cost assignment.

1.1.1 Cost Model

Software Execution Cost (s_i): Represents the estimated execution time when task i runs on a general-purpose processor. These costs are uniformly sampled in the range $[1, 100]$ to simulate varying computational complexity across different tasks.

Hardware Execution Cost (h_i): Represents the execution time when task i is implemented as a dedicated hardware accelerator. The hardware cost is modeled as:

$$h_i = \max(0.1, \mathcal{N}(k \cdot s_i, (l \cdot k \cdot s_i)^2)) \tag{1}$$

where $k > 0$ is the hardware scale factor and $l > 0$ is the hardware scale variance parameter. The scale factor k determines the relative performance advantage of hardware implementation. During our implementation, we set $k < 1$ to indicate faster hardware execution time in general compared to its software execution counterpart. The variance parameter l controls the variability in hardware costs relative to software costs, with smaller values maintaining a stronger correlation between s_i and h_i.

Hardware Area Cost (a_i): Each hardware implementation incurs an area cost uniformly distributed in the range $[1, A_{max}]$, where A_{max} represents the maximum area requirement for any single task. To impose area constraint, we define the area constraint ratio α. Thus, any partitioning solution $\mathcal{P}$ should

satisfy the following constraint $\sum_{i \in H_{\mathcal{P}}} a_i \leq \alpha \cdot \sum_i a_i$, where $H_{\mathcal{P}}$ is the set of tasks assigned to hardware as per partitioning $\mathcal{P}$.

Communication Cost ($c_{i,j}$): Represents the overhead incurred when data flows between tasks i and j assigned to different processing units. Communication costs are uniformly distributed in the range $[0, 2\mu \cdot s_{max}]$, where $s_{max} = \max_i(s_i)$ and μ is the communication scale factor. Higher values of μ increase the penalty for cross-platform communication, making partitioning decisions more critical for overall performance.

1.1.2 Experimental Parameters

The experimental evaluation employs the following parameter settings:

- **Hardware Scale Factor (k):** Controls the relative hardware-software performance ratio
- **Hardware Scale Variance (l):** Determines cost correlation variability
- **Communication Scale Factor (μ):** Governs inter-partition communication overhead
- **Area Constraint (α):** Fraction of total hardware area available for allocation

Configuration parameters are specified via YAML files, enabling systematic parameter sweeps and reproducible experimental conditions. Each experiment generates comprehensive logging and saves intermediate results including task graph instances, partition assignments, and performance metrics.

1.1.3 Objective Functions

The quality of the solution $\mathcal{P}$ is evaluated against six separate objective functions.

ARATO: The total cost, including execution costs and communication overhead:

$$C_{total} = \sum_{i \in S_{\mathcal{P}}} s_i + \sum_{i \in H_{\mathcal{P}}} h_i + \sum_{(i,j) \in E_{\mathcal{P}}} c_{i,j} \tag{2}$$

where $S_{\mathcal{P}}, H_{\mathcal{P}}$ and $E_{\mathcal{P}}$ are software-assigned tasks, hardware-assigned tasks, and edges between tasks on different platforms induced by partitioning $\mathcal{P}$, respectively.

(ZM: This is a proxy cost of the actual cost as there are parallelism. Also serves as an upper bound. Similar cost was used in [1])

OPTM (One Possible Topological makespan): While tasks assigned to software execute sequentially, hardware tasks can execute in parallel (ZM: cite a reference), given the set of the precedence tasks have completed execution.

(ZM: This requires knowing the scheduling. How to write this part– needs discussion.)

Here, we run the meta-heuristics to optimize one possible scheduling by choosing the lexicographically smallest possible topological ordering of the tasks at every point when there are multiple ready nodes, giving us one possible scheduling. Optimizing this objective function is more computationally extensive than the partition cost as it now requires one to run a $\mathcal{O}(n^2)$ heuristic (ZM: check with Rounak if the complexity is correct) to calculate the cost function during optimization.

Area constraint violations incur a penalty cost of 10^9 to ensure feasible solutions, effectively treating the problem as a constrained optimization task.

CA (Communication-Aware): (ZM: write about comm-aware heuristic)

CPL (Critical Path List Scheduling): (ZM: write about this heuristic)

ED (Event-driven): (ZM: write about event-driven heuristic)

SPT (Shortest processing time): (ZM: write about this heuristic)

Note that, during evaluations, all the solutions are evaluated using the OPTM strategy.

(ZM: this probably needs to be updated)

1.2 Baseline Methods

1.2.1 Baseline Methods

To provide a comprehensive performance evaluation, we implement several baseline and state-of-the-art meta-heuristic optimization methods. The baseline methods span multiple optimization paradigms, enabling us to establish comprehensive performance benchmarks.

Random Assignment: A baseline method that randomly assigns tasks to hardware or software platforms with uniform probability. Each task i is assigned to hardware with probability 0.5, providing a lower bound for algorithm performance and serving as a sanity check for more sophisticated methods.

Greedy Heuristic: A based on the fractional knapsack problem approach. For each task i, we calculate the gain-per-area ratio as $(s_i - h_i)/a_i$ and sort tasks in descending order of this ratio. Then, the tasks are greedily assigned to hardware until the area constraint is violated, and the remaining tasks are assigned to software.

Particle Swarm Optimization (PSO): The standard PSO algorithm [3] where particles represent continuous-valued solutions in $[0, 1]^n$ space, n being the number of tasks. The particle values are converted to binary representation using a sigmoid transformation of the form $1/(1 + e^{-x})$, where x is the original particle value. The PSO has three parameters: inertia weight w, cognitive coefficient c_1, and social coefficient c_2.

Discrete Binary PSO (DBPSO): A discrete variant of PSO specifically designed for binary optimization problems [4]. Unlike continuous PSO, DBPSO operates directly in binary space, making it naturally suited for hardware-software assignment decisions.

Comprehensive Learning PSO (CLPSO): An enhanced PSO variant [6] where particles learn from different exemplars for different dimensions, promot-

Table 1: Parameter used for baseline methods. Numbers in bracket indicate changed values for OPTM objective function

Algorithm	**Parameter**	**Value**
PSO	Cognitive coefficient (c_1)	0.575
	Social coefficient (c_2)	0.1
	Inertia weight (w)	1.05
	Population size	500
	Iterations	1000 (*10)
DBPSO	Cognitive coefficient (c_1)	0.575
	Social coefficient (c_2)	0.1
	Inertia weight (w)	1.05
	Number of neighbors (k)	4
	Minkowski p-norm ($p \in \{1, 2\}$)	2
	Population size	500
	Iterations	1000 (*10)
CLPSO	Comprehensive Learning Rate (c)	1
	Population size	500
	Iterations	1000 (*200)
CCPSO	Population size	500
	Iterations	1000 (*200)
	Group sizes	[5, 10, 20]
GL25	Iterations	10000 (*2000)
	Population size	500
SHADE	Iterations	10000 (*2000)
	Population size	500
JADE	Iteartions	10000 (*2000)
	Population size	500
ESA	Iterations	10000 (*1000)
Random	Number of samples	500
	Assignment probability	0.5

Table 2: % Improvment of MIP over various meta heuristic methods for $k = 0.1, \alpha = 0.5$

opt_strategy	ccpso	clpso	dbpso	esa	gl25	jade	pso	shade
ARATO	50.87	50.87	50.87	20.0	20.63	50.87	49.31	50.87
CA	42.55	42.83	40	-4.16	-7.05	34.68	33.38	34.37
CPL	43.94	42.5	40.14	6.35	13.55	37.18	34.39	38.6
ED	43.15	42.24	39.29	11.5	1.13	35.25	34.75	37.78
OPTM	43.27	43.21	41.91	43.05	9.35	40.27	46.38	39.29
SPT	44.09	43.44	37.56	7.94	7.56	35.45	45.30	37.45

ing population diversity and reducing premature convergence. Instead of using the global best to update velocity along all dimensions, it considers the personal best of selected particles to update the velocity along each dimension.

Cooperative Co-evolutionary PSO (CCPSO): A divide-and-conquer approach [5] that decomposes the optimization problem into smaller subproblems, with separate swarms optimizing different variable groups.

Success-History based Adaptive Differential Evolution (SHADE): An adaptive variant of Differential Evolution [8] that maintains historical information about successful parameter settings.

Adaptive Differential Evolution with Optional External Archive (JADE): Another adaptive DE variant [9] that uses an optional external archive to store recently discarded solutions and employs parameter adaptation mechanisms.

Genetic Algorithm GL25: Global and local genetic algorithm [2] where 25% of function evaluations are used for global search and remaining 75% percentage are used for local search.

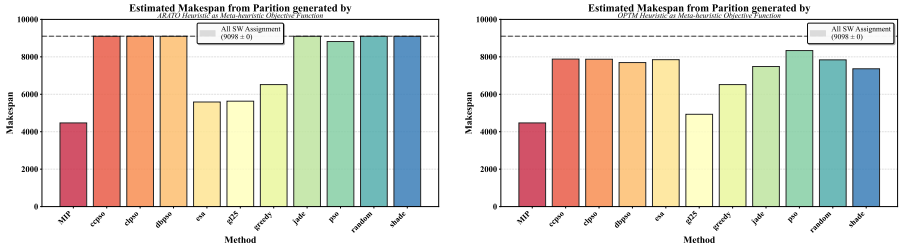
Enhanced Simulated Annealing (ESA): An improved simulated annealing algorithm [7] adopting a random decomposition strategy to alleviate the curse of dimensionality for large-scale black-box optimization.

1.3 Results

1.3.1 SqueezeNet, $k = 0.1, \alpha = 0.5$

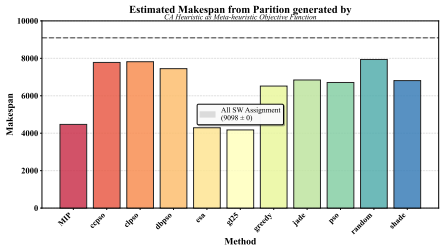
Figure 1 shows the performance of various methods with respect to various optimization strategies (Random and greedy does not have any). Table 2 presents the percentage improvement of the Mixed Integer Programming (MIP) solution over various meta-heuristic methods for hardware scale factor $k = 0.1$ and area constraint $\alpha = 0.5$. The results reveal significant variations in performance across different optimization strategies and meta-heuristic algorithms.

Drawback of ARATO strategy The ARATO optimization strategy exhibits the poorest performance for the meta heuristics, with MIP outperforming most meta-heuristics by approximately 50%, except for ESA (20.0%) and GL25

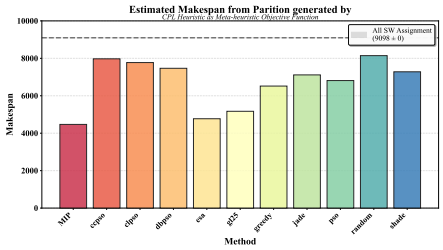


(a) ARATO Heuristic

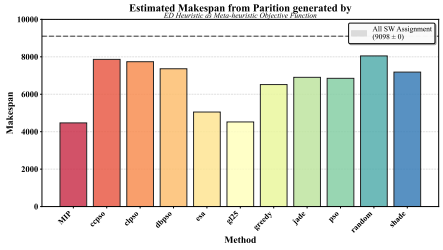
(b) OPTM Heuristic



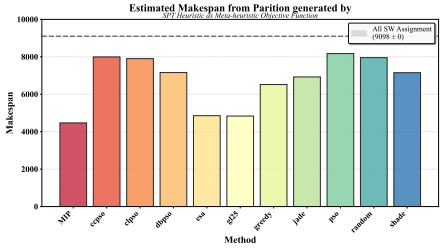
(c) CA heuristic



(d) CPL heuristic



(e) ED heuristic



(f) SPT heuristic cost

Figure 1: Comparison of Various Baseline MetaHeuristics on different optimization functions

(20.63%). This substantial gap suggests that ARATO-based meta-heuristics struggle to find near-optimal solutions for the given problem constraints.

Best Performing Meta-Heuristics: Across all the optimization strategies, GL25 and ESA consistently demonstrate the smallest performance gaps relative to MIP. Notably, for the CA (Communication-Aware) strategy, both methods achieve superior evaluation metrics compared to MIP. (ZM: This is due to MIP jointly solving partitioning and scheduling, and the evaluation function takes a specific scheduling. Thus, the superiority of the MIP may be overlooked here). This suggests that for communication-intensive objectives, meta-heuristics can explore solution spaces more effectively than exact methods within practical time constraints. –(ZM: May not be necessarily true, but given the current nature of evaluation, this is the best I can put)

Strategy-Specific Observations:

- **ARATO:** Shows poor performance across PSO variants (PSO, DBPSO, CCPSO, CLPSO, JADE, SHADE), suggesting these methods converge to similar suboptimal regions for this strategy.
- **CA (Communication-Aware):** Exhibits the most varied performance, with GL25 and ESA achieving superior solutions to MIP while PSO variants lag by 33–43%. This indicates that genetic algorithms and simulated annealing better capture the communication cost structure.
- **OPTM (Makespan Optimization):** PSO has the largest improvement gap (46.38%), while GL25 shows exceptional performance with only 9.35% gap, demonstrating the effectiveness of genetic approaches for scheduling-oriented objectives.
- **CPL, ED, and SPT:** These strategies show similar margins, with GL25 and ESA consistently performing better than PSO-based approaches.

Algorithm Family Comparison:

- *PSO Variants* (PSO, DBPSO, CCPSO, CLPSO): Demonstrate relatively consistent performance within each strategy, with optimality gaps ranging from 33–50%. CCPSO and CLPSO variations show minimal differentiation from standard PSO.
- *Differential Evolution* (JADE, SHADE): Perform comparably to PSO variants, with JADE generally achieving slightly better results than SHADE across most strategies.
- *Genetic Algorithm* (GL25): Stands out as the most competitive metaheuristic, achieving the smallest gaps across five of six strategies.
- *Simulated Annealing* (ESA): Shows strong performance particularly in CA and CPL strategies.

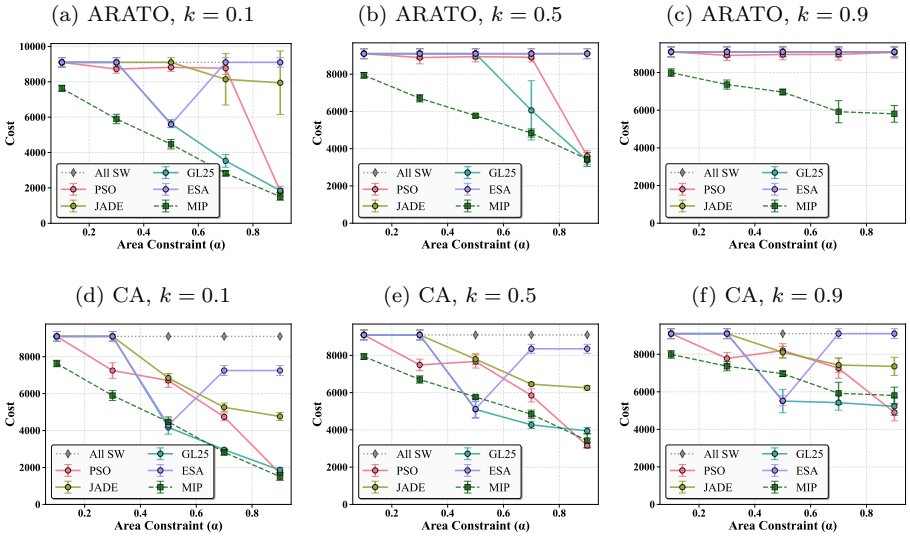
Our first analysis reveals that GL25 and ESA consistently achieve superior performance across all objective functions, establishing it as the most robust methods for this particular task graph configuration. (ZM: Note that ESA,GL25 have higher iterations than many other (except SHADE, JADE))

(ZM: Before proceeding more, we need to decide the following.

1. All experimental configurations – number of environment parameters to evaluate against, how many run per each config.
2. task graphs – how many taskgraphs to evaluate against
3. evaluation functions – what function to use for evaluation

)

Figure 2: Sensitivity analysis of meta-heuristic performance across optimization strategies, hardware scale factors and area constraint



2 Sensitivity Analysis

This section presents a detailed sensitivity analysis of four meta-heuristic algorithms (PSO, JADE, GL25, ESA) against two baselines (MIP (expected best), naive all-software (expected worst)) across two optimization strategies and various hardware scale factors ($k \in \{0.1, 0.5, 0.9\}$) and area constraints ($\alpha \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$).

2.1 Optimization Strategy: ARATO

At $k = 0.1$ where hardware assignment provides a $10\times$ speedup on average than the corresponding software assignment, the meta-heuristics exhibit significant differences in their performances when they use ARATO optimization strategy. GL25 emerges as the clear winner, achieving competitive performance with respect to the MIP solution. However, PSO is able to find better solution compared to GL25 for tighter area constraints. ESA exhibits unusual behaviour, performing well at intermediate area constraints but reverting to naive performance for more relaxed constraints, suggesting premature convergence or acceptance of local optima. JADE catastrophically fails for this optimization strategy at $k = 0.1$, maintaining costs near the naive assignment even at high area budgets (7,942.28 at $\alpha = 0.9$).

As hardware advantage diminishes to $k = 0.5$, GL25 maintains the best performance. However, it is only able to maintain acceptable margin with MIP for relaxed area constraint. This suggests GL25 excels specifically when sub-

stantial hardware resources are available. The collapse of other meta-heuristic performance at $k = 0.5$ suggests ARATO's optimization landscape becomes dramatically more disadvantegeous as hardware advantage diminishes.

At $k = 0.9$ where hardware provides only marginal speedup, meta-heuristic performance completely collapses for ARATO optimization strategy. This universal failure suggests the limited applicability of this optimization strategy when hardware allocation provides negligible benefit.

2.2 Optimization Strategy: CA

At $k = 0.1$, the CA strategy proves to be more effective compared to ARATO. While GL25 is again the best meta-heuristic, the performance gap with MIP diminishes by a lot, demonstrating that genetic algorithms can exploit this strategy better than ARATO. PSO achieves strong performance at relaxed area constraint. ESA again shows anomalous behaviour, suggesting a fundamental limitation of the meta-heuristic itself in the current problem domain, rather than an issue with the optimization strategy. The struggle of JADE presists, though it performs notably better under CA than ARATO.

At moderate hardware advantage ($k = 0.5$), CA strategy maintains better meta-heuristic effectiveness than ARATO at the same scale factor. GL25 achieves an average of 6.04% performance gap from MIP. This represents a fundamental difference from ARATO; CA strategy maintains effective meta-heuristic exploration even as hardware advantages diminish. PSO achieves the second best performance, followed by ESA, which exhibits its characteristic intermediate-optimum behaviour but struggles at extreme constraints. JADE continues to underperform with a better margin than ARATO, suggesting it navigates the landscape more effectively under CA optimization strategy.

Even at minimal hardware advantage, CA strategy maintains meta-heuristic viability. The maintained meta-heuristic effectiveness at $k = 0.9$ under CA strategy (versus complete collapse under ARATO) demonstrates fundamental differences in objective function landscapes.

2.3 Fundamental Insights on Optimization Landscape Structure

The dramatic performance variations across strategies and hardware scales reveal fundamental insights.

References

[1] Arató, P., Mann, Z. Á., and Orbán, A. Algorithmic aspects of hardware/software partitioning. *ACM Transactions on Design Automation of Electronic Systems (TODAES) 10*, 1 (2005), 136–156.

[2] García-Martínez, C., Lozano, M., Herrera, F., Molina, D., and Sánchez, A. M. Global and local real-coded genetic algorithms based on

parent-centric crossover operators. *European journal of operational research 185*, 3 (2008), 1088–1113.

[3] Kennedy, J., and Eberhart, R. Particle swarm optimization. In *Proceedings of ICNN'95-international conference on neural networks* (1995), vol. 4, ieee, pp. 1942–1948.

[4] Kennedy, J., and Eberhart, R. C. A discrete binary version of the particle swarm algorithm. In *1997 IEEE International conference on systems, man, and cybernetics. Computational cybernetics and simulation* (1997), vol. 5, ieee, pp. 4104–4108.

[5] Li, X., and Yao, X. Cooperatively coevolving particle swarms for large scale optimization. *IEEE Transactions on Evolutionary Computation 16*, 2 (2011), 210–224.

[6] Liang, J. J., Qin, A. K., Suganthan, P. N., and Baskar, S. Comprehensive learning particle swarm optimizer for global optimization of multimodal functions. *IEEE transactions on evolutionary computation 10*, 3 (2006), 281–295.

[7] Siarry, P., Berthiau, G., Durdin, F., and Haussy, J. Enhanced simulated annealing for globally minimizing functions of many-continuous variables. *ACM Transactions on Mathematical Software (TOMS) 23*, 2 (1997), 209–228.

[8] Tanabe, R., and Fukunaga, A. Success-history based parameter adaptation for differential evolution. In *2013 IEEE congress on evolutionary computation* (2013), IEEE, pp. 71–78.

[9] Zhang, J., and Sanderson, A. C. Jade: adaptive differential evolution with optional external archive. *IEEE Transactions on evolutionary computation 13*, 5 (2009), 945–958.

A Vanilla Partitioning Problem

A.1 Problem Definition

We are given a graph $G = (V, E)$ with n nodes and m edges. Each node has two node weights h_i and s_i to denote hardware and software costs. The edges denote communication between the nodes and have associated communication cost c_{ij} for the edge e connecting nodes i and j. The aim is to partition the nodes V into non-overlapping sets V_S and V_H denoting software and hardware nodes respectively; $V = V_H \cup V_S$ and $V_H \cap V_S = \emptyset$. We denote the set of edges between the two sets by $E_{\mathcal{P}}$. Note that we will only consider communication cost for these edges which are between hardware and software nodes. The hardware cost for a partition $\mathcal{P}$ is defined as sum of hardware cost for the nodes belonging to the hardware nodes $H_{\mathcal{P}} = \sum_{v \in V_H} h_v$. Similarly, the software cost for the partition is $S_{\mathcal{P}} = \sum_{v \in V_S} s_v$. The communication cost is defined as $C_{\mathcal{P}} = \sum_{e:(i,j) \in E_{\mathcal{P}}} c_{ij}$.

A.2 Optimization Formulation

We define binary variable x_i for each node $i \in V$; $x_i = 1$ if node i is a software node in partitioning $\mathcal{P}$, else $x_i = 0$ for a hardware node. Additionally, we denote the edge weighted adjacency matrix for the graph G to be $\mathbf{C}$, where the element along the i^{th} row and j^{th} column C_{ij} is

$$C_{ij} = \begin{cases} c_{ij}, \text{ if } e : (i,j) \in E \\ 0, \text{ otherwise} \end{cases}$$

Note that $\mathbf{C}$ is symmetric. The communication cost for the partition $\mathcal{P}$ is

$$C_{\mathcal{P}} = \sum_{e:(i,j)\in E_{\mathcal{P}}} c_{ij} = \sum_{i\in V}\sum_{j\in V} x_i C_{ij}(1 - x_j)$$

We can stack the entries h_i, s_i and x_i in n-length vectors $\mathbf{h}$, $\mathbf{s}$ and $\mathbf{x}$ respectively. For the partition $\mathcal{P}$, the hardware cost is $H_{\mathcal{P}} = \mathbf{h}^T(\mathbf{1} - \mathbf{x})$, the software cost is $S_{\mathcal{P}} = \mathbf{s}^T\mathbf{x}$, and the communication cost $C_{\mathcal{P}} = (\mathbf{1} - \mathbf{x})\,\mathbf{C}\mathbf{x}$. The hardware-software partitioning problem can be stated as

$$\begin{aligned} \mathcal{P}^\star = \arg\min\ & H_{\mathcal{P}} \\ \text{s.t. } & S_{\mathcal{P}} + C_{\mathcal{P}} \le R_0 \end{aligned}$$

which we can write as

$$\mathbf{x}^\star = \arg\min\ \mathbf{h}^T(\mathbf{1} - \mathbf{x}) \tag{3a}$$

$$\text{s.t. } \mathbf{s}^T\mathbf{x} + (\mathbf{1} - \mathbf{x})^T\mathbf{C}\mathbf{x} \le R_0 \tag{3b}$$

$$x_i \in \{0,1\},\ \forall i = 1, 2, \cdots, n \tag{3c}$$

Relaxation 1. *We relax the integer constraint $x_i \in \{0,1\}$ to $0 \le x_i \le 1,\ \forall i = 1, 2, \cdots, n$.*

The relaxed optimization problem is a quadratic constrained linear problem

$$\mathbf{x}^\star = \arg\min\ \mathbf{h}^T(\mathbf{1} - \mathbf{x}) \tag{4a}$$

$$\text{s.t. } (\mathbf{s} + \mathbf{C1})^T\mathbf{x} - \mathbf{x}^T\mathbf{C}\mathbf{x} \le R_0 \tag{4b}$$

$$\mathbf{0} \le \mathbf{x} \le \mathbf{1} \tag{4c}$$

Note that $\mathbf{C} \succeq 0$, and hence, the constraint $(\mathbf{s} + \mathbf{C1})^T\mathbf{x} - \mathbf{x}^T\mathbf{C}\mathbf{x} \le R_0$ is non-convex. We now define $(n+1) \times (n+1)$ matrices

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}\mathbf{x}^T & \mathbf{x} \\ \mathbf{x}^T & 1 \end{bmatrix} \text{ and } \mathbf{H} = \begin{bmatrix} 0 & \frac{\mathbf{h}}{2} \\ \frac{\mathbf{h}^T}{2} & 0 \end{bmatrix}$$

to pose the above problem as a semidefinite program (SDP). Using the linearity and cyclic properties of *trace* of a matrix, we have

$$\mathrm{Tr}\,(\mathbf{H}\mathbf{X}) = \mathbf{h}^T\mathbf{x} \tag{5a}$$

$$\mathrm{Tr}\left(\begin{bmatrix} \mathbf{C} & -\frac{\mathbf{s}+\mathbf{C1}}{2} \\ -\frac{(\mathbf{s}+\mathbf{C1})^T}{2} & R_0 \end{bmatrix}\mathbf{X}\right) = \mathbf{x}^T\mathbf{C}\mathbf{x} - (\mathbf{s} + \mathbf{C1})^T\mathbf{x} + R_0 \tag{5b}$$

Additionally, we want to add some more constraints to make the problem more tight. These include

$$\forall i = 1, 2, \cdots, n \quad 0 \leq x_i \leq 1 \quad \Rightarrow 0 \leq \mathbf{e}_i^T \mathbf{x} \leq 1 \tag{6a}$$

$$\forall i = 1, 2, \cdots, n \quad X_{ii} = x_i \quad \Rightarrow \mathbf{x}^T \mathbf{E}_{ii} \mathbf{x} = \mathbf{e}_i^T \mathbf{x} \tag{6b}$$

$$\forall i, j = 1, 2, \cdots, n \quad X_{ij} \leq x_i \quad \Rightarrow \mathbf{x}^T \mathbf{E}_{ij} \mathbf{x} \leq \mathbf{e}_i^T \mathbf{x} \tag{6c}$$

$$\forall i, j = 1, 2, \cdots, n \quad X_{ij} \leq x_j \quad \Rightarrow \mathbf{x}^T \mathbf{E}_{ij} \mathbf{x} \leq \mathbf{e}_j^T \mathbf{x} \tag{6d}$$

$$\forall i, j = 1, 2, \cdots, n \quad X_{ij} \geq 0 \quad \Rightarrow \mathbf{x}^T \mathbf{E}_{ij} \mathbf{x} \geq 0 \tag{6e}$$

Here, $\mathbf{E}_{ij}$ denotes a $n \times n$ matrix with all zeros, except the element along the i^{th} row and j^{th} column being 1, and the vector $\mathbf{e}_i$ denotes the i^{th} column of the $n \times n$ identity matrix. Using the linearity and cyclic properties of *trace* of a matrix, we have the following identities

$$\mathrm{Tr}\left(\begin{bmatrix} \mathbf{0} & \frac{\mathbf{e}_i}{2} \\ \frac{\mathbf{e}_i^T}{2} & 0 \end{bmatrix} \mathbf{X} \right) = \mathbf{e}_i^T \mathbf{x} \tag{7a}$$

$$\mathrm{Tr}\left(\begin{bmatrix} \mathbf{E}_{ij} & -\frac{\mathbf{e}_i}{2} \\ -\frac{\mathbf{e}_i^T}{2} & 0 \end{bmatrix} \mathbf{X} \right) = \mathbf{x}^T \mathbf{E}_{ij} \mathbf{x} - \mathbf{e}_i^T \mathbf{x} \tag{7b}$$

The SDP problem is

$$\text{minimize } \mathbf{h}^T \mathbf{1} - \mathrm{Tr}(\mathbf{H}\mathbf{X}) \tag{8a}$$

$$\text{subject to } \mathrm{Tr}\left(\begin{bmatrix} \mathbf{C} & -\frac{\mathbf{s}+\mathbf{C1}}{2} \\ -\frac{(\mathbf{s}+\mathbf{C1})^T}{2} & R_0 \end{bmatrix} \mathbf{X} \right) \geq 0 \tag{8b}$$

$$0 \leq \mathrm{Tr}\left(\begin{bmatrix} \mathbf{0} & \frac{\mathbf{e}_i}{2} \\ \frac{\mathbf{e}_i^T}{2} & 0 \end{bmatrix} \mathbf{X} \right) \leq 1, \quad \mathrm{Tr}\left(\begin{bmatrix} \mathbf{E}_{ii} & -\frac{\mathbf{e}_i}{2} \\ -\frac{\mathbf{e}_i^T}{2} & 0 \end{bmatrix} \mathbf{X} \right) = 0, \quad \forall i \tag{8c}$$

$$\mathrm{Tr}\left(\begin{bmatrix} \mathbf{E}_{ij} & -\frac{\mathbf{e}_i}{2} \\ -\frac{\mathbf{e}_i^T}{2} & 0 \end{bmatrix} \mathbf{X} \right) \leq 0, \quad \mathrm{Tr}\left(\begin{bmatrix} \mathbf{E}_{ij} & -\frac{\mathbf{e}_j}{2} \\ -\frac{\mathbf{e}_j^T}{2} & 0 \end{bmatrix} \mathbf{X} \right) \leq 0 \quad \forall i, j \tag{8d}$$

$$\mathrm{Tr}\left(\begin{bmatrix} \mathbf{E}_{ij} & \mathbf{0} \\ \mathbf{0}^T & 0 \end{bmatrix} \mathbf{X} \right) \geq 0 \quad \forall i, j, \quad \mathbf{X} \succeq 0 \tag{8e}$$

$$\mathrm{rank}(\mathbf{X}) = 1 \tag{8f}$$

B Scheduling Constrained Partitioning Problem

B.1 Problem Definition

We are given a graph $G = (V, E)$ with n nodes and m edges. Each node has weights h_i and s_i to denote hardware and software execution times. Also, each node has an associated hardware area cost a_i. The edges denote communication between the nodes and have associated communication cost c_{ij} for the edge e connecting nodes i and j. The communication delay is taken into account

only when the adjacent nodes belong to different partitions. The aim is to partition the nodes V into non-overlapping sets V_S and V_H denoting software and hardware nodes respectively; $V = V_H \cup V_S$ and $V_H \cap V_S = \emptyset$. We denote the set of edges between the two sets by $E_{\mathcal{P}}$. Additionally, we need to ensure that software nodes are executed sequentially, while hardware nodes can execute in parallel if their dependent nodes are executed. Our objective is to minimize the overall execution time given a limited available area for hardware nodes.

B.2 Optimization Formulation

Decision variables. We define binary variable $x_i = 0$ if node i is assigned to hardware partition, $x_i = 1$ if assigned to software partition. We also define binary variable z_{ij} for each edge $e := (i, j) \in E$connecting nodes i and j. We denote $z_{ij} = 1$ if the nodes i and j belong to different partitions. Let t_i and f_i denote the start and end time of the node i. The total execution time of the task graph is denoted by T. We also define binary variable y_{ij} for each pair of nodes i, j, $i \neq j$. This binary variable is used to ensure that if the pair of nodes i, j are assigned to software partition, they need to be executed sequentially; i.e., if $y_{ij} = 1$, then node i precedes node j, and for $y_{ij} = 0$, the opposite happens. However, we need to ensure that this sequential constraint is activated for only a pair of software nodes.

Hardware area constraint. The overall hardware area is constrained by the maximum allowable hardware area $A_{\max}$.

$$\sum_{i=1}^{n} a_i \left(1 - x_i\right) \leq A_{\max} \tag{9}$$

Binary edge variable. The binary variable z_{ij} and the partition variable x_i, x_j corresponding to the adjacent nodes are related by

$$z_{ij} \geq x_i - x_j, \qquad \forall \left(i, j\right) \in E \tag{10a}$$
$$z_{ij} \geq x_j - x_i, \qquad \forall \left(i, j\right) \in E \tag{10b}$$
$$z_{ij} \leq x_i + x_j, \qquad \forall \left(i, j\right) \in E \tag{10c}$$
$$z_{ij} \leq 2 - x_i - x_j, \qquad \forall \left(i, j\right) \in E \tag{10d}$$

Execution time constraints. The execution time of each node is denoted

$$T \geq f_i, \qquad \forall i \in V \tag{11a}$$
$$f_i = t_i + h_i \left(1 - x_i\right) + s_i x_i, \qquad \forall i \in V \tag{11b}$$
$$f_i + c_{ij} z_{ij} \leq t_j, \qquad \forall \left(i, j\right) \in E \tag{11c}$$

Sequential execution of software nodes. We define the following pair of constraints with M being a large positive number.

$$f_i \leq t_j + M \left(1 - y_{ij}\right) + M \left(2 - x_i - x_j\right), \qquad \forall i, j \in V, i \neq j \tag{12a}$$
$$f_j \leq t_i + M y_{ij} + M \left(2 - x_i - x_j\right), \qquad \forall i, j \in V, i \neq j \tag{12b}$$

Note that, the constraints are activated only when $x_i = x_j = 1$ or the nodes i, j are both software nodes, when the term $2 - x_i - x_j = 0$. For other cases, the constraints are not active since M is sufficiently large. Once the constraints are activated, the value of y_{ij} denotes the precedence of the software node operations.

The overall optimization problem is

$$\text{minimize } T \tag{13a}$$

$$\text{subject to } (9), (10), (11), (12) \tag{13b}$$